ANN for Wine Quality Prediction

Using the provided wine quality dataset, develop an Artificial Neural Network (ANN) model to classify wine quality based on its chemical features. The dataset contains 1143 samples with 12 input features.

Your task is to:

1. Preprocess the dataset and split it into training and testing sets.
2. Build and train an ANN model using appropriate activation functions and optimizer.
3. Train the model using mini-batch gradient descent (batch size = 32) and evaluate its performance.
4. Analyze the training and validation results to check convergence.
5. Experiment with different activation functions, optimizers, and dropout regularization, and observe their effect on model accuracy and overfitting.
6. Explain your observations based on the training results.

Dataset used: Wine Quality Dataset

Dataset link: https://www.kaggle.com/datasets/yasserh/wine-quality-dataset

Libraries used: Python + TensorFlow/Keras

Step 1: Import Required Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.optimizers import Adam
```

We import libraries required for:

NumPy → numerical computations

Pandas → dataset handling

Matplotlib → visualization

train_test_split → dataset splitting

StandardScaler → feature normalization

TensorFlow/Keras → building neural network

Q1: What is Pandas used for?

A: It loads and manipulates datasets.

Q2: What is TensorFlow used for?

A: Building and training neural networks.

Step 2: Load the Dataset

```
from google.colab import files
uploaded = files.upload()
```

Choose Files WineQT.csv

WineQT.csv(text/csv) - 78057 bytes, last modified: 3/9/2026 - 100% done

```
Saving WineQT.csv to WineQT.csv
```

```
data = pd.read_csv('WineQT.csv')
data.head()
data.info()
data.shape
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1143 entries, 0 to 1142
```

```
Data columns (total 13 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   fixed acidity         1143 non-null   float64
 1   volatile acidity      1143 non-null   float64
 2   citric acid           1143 non-null   float64
 3   residual sugar        1143 non-null   float64
 4   chlorides             1143 non-null   float64
 5   free sulfur dioxide   1143 non-null   float64
 6   total sulfur dioxide  1143 non-null   float64
 7   density               1143 non-null   float64
 8   pH                    1143 non-null   float64
 9   sulphates             1143 non-null   float64
 10  alcohol               1143 non-null   float64
 11  quality               1143 non-null   int64
 12  Id                    1143 non-null   int64
dtypes: float64(11), int64(2)
memory usage: 116.2 KB
(1143, 13)
```

This loads the dataset into a DataFrame and displays the first few rows.

Q3: What does data.head() do?

A: Displays the first 5 rows of the dataset.

Q4: Why should we inspect data before training a model?

A: To understand feature types and detect issues like missing values.

Step 3: Define Features and Target

```
X = data.drop('quality', axis=1)
y = data['quality']

y = (y > 5).astype(int)
```

We separate:

X ⟶ input features

y ⟶ target variable

Wine quality is converted into:

Quality Score Class

≤5 Low quality

Q5: Why separate features and target?

A: Machine learning models learn patterns from features to predict the target.

```
print(X.shape)
print(y.shape)
```

```
(1143, 12)
(1143,)
```

Step 4: Split Dataset

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Dataset is divided into:

Training- teach model

Testing - evaluate model

Step 5: Normalize Data

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Feature scaling ensures all variables are in a similar numerical range, which helps neural networks train efficiently.

Step 6: Build the Neural Network

```
model = Sequential()

model.add(Dense(16, activation='relu', input_shape=(12,)))
model.add(Dense(8, activation='relu'))

model.add(Dropout(0.2))

model.add(Dense(1, activation='sigmoid'))
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Architecture:

Layer Neurons

Input 11

Hidden 1 16

Hidden 2 8

Output 1

Dropout randomly disables neurons during training to reduce overfitting.

Q6: Why do we use hidden layers?

A: To capture complex nonlinear relationships.

Step 7: Compile the Model

```
model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='binary_crossentropy',
    metrics=['accuracy']
)
```

Defines:

optimizer → weight update method

loss → error calculation

metrics → performance measure

Q7. Why use binary cross entropy?

A: Because the problem is binary classification.

Step 8: Train the Model

```
history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

```
Epoch 1/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 2s 14ms/step - accuracy: 0.5597 - loss: 0.7016 - val_accuracy: 0.6393 - val_loss: 0.6349
Epoch 2/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.5655 - loss: 0.6819 - val_accuracy: 0.6557 - val_loss: 0.6173
Epoch 3/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.5978 - loss: 0.6519 - val_accuracy: 0.6721 - val_loss: 0.6051
Epoch 4/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6352 - loss: 0.6154 - val_accuracy: 0.6885 - val_loss: 0.5941
Epoch 5/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6342 - loss: 0.6167 - val_accuracy: 0.6776 - val_loss: 0.5865
Epoch 6/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6574 - loss: 0.5949 - val_accuracy: 0.6831 - val_loss: 0.5778
Epoch 7/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6553 - loss: 0.5939 - val_accuracy: 0.6940 - val_loss: 0.5684
```

```
Epoch 8/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6835 - loss: 0.5867 - val_accuracy: 0.7104 - val_loss: 0.5596
Epoch 9/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.6840 - loss: 0.5793 - val_accuracy: 0.7158 - val_loss: 0.5517
Epoch 10/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6765 - loss: 0.5884 - val_accuracy: 0.7213 - val_loss: 0.5450
Epoch 11/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6796 - loss: 0.5784 - val_accuracy: 0.7377 - val_loss: 0.5407
Epoch 12/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7024 - loss: 0.5610 - val_accuracy: 0.7322 - val_loss: 0.5367
Epoch 13/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7180 - loss: 0.5428 - val_accuracy: 0.7322 - val_loss: 0.5354
Epoch 14/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7087 - loss: 0.5606 - val_accuracy: 0.7432 - val_loss: 0.5321
Epoch 15/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7355 - loss: 0.5475 - val_accuracy: 0.7322 - val_loss: 0.5287
Epoch 16/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7040 - loss: 0.5659 - val_accuracy: 0.7432 - val_loss: 0.5307
Epoch 17/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7422 - loss: 0.5464 - val_accuracy: 0.7432 - val_loss: 0.5255
Epoch 18/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7271 - loss: 0.5244 - val_accuracy: 0.7432 - val_loss: 0.5234
Epoch 19/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7729 - loss: 0.5066 - val_accuracy: 0.7432 - val_loss: 0.5237
Epoch 20/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7552 - loss: 0.5364 - val_accuracy: 0.7432 - val_loss: 0.5252
Epoch 21/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7404 - loss: 0.5233 - val_accuracy: 0.7322 - val_loss: 0.5216
Epoch 22/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7527 - loss: 0.5465 - val_accuracy: 0.7377 - val_loss: 0.5207
Epoch 23/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7570 - loss: 0.4987 - val_accuracy: 0.7377 - val_loss: 0.5179
Epoch 24/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7325 - loss: 0.5347 - val_accuracy: 0.7377 - val_loss: 0.5158
Epoch 25/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7423 - loss: 0.5411 - val_accuracy: 0.7322 - val_loss: 0.5172
Epoch 26/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7569 - loss: 0.5159 - val_accuracy: 0.7213 - val_loss: 0.5182
Epoch 27/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7626 - loss: 0.5115 - val_accuracy: 0.7322 - val_loss: 0.5161
Epoch 28/50
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7275 - loss: 0.5374 - val_accuracy: 0.7377 - val_loss: 0.5145
Epoch 29/50
```

Training involves:

Forward propagation

Loss calculation

Backpropagation

Weight updates

Q8: What is an epoch?

Epoch 1/50

Epoch = one complete pass through the entire training dataset.

Your model trained 50 times over the dataset.

Example:

Dataset size = 1000 samples

Epoch = 1 ⟶ model sees all 1000 samples once

Epoch = 2 ⟶ model sees them again

dataset shape:

X = (1143, 12) y = (1143,)

Meaning 1143 samples with 12 features.

Train–Test Split (20% test)

Test = 1143 × 0.2 ≈ 229

Train = 1143 − 229 = 914

Validation Split (20% of training)

Validation = 914 × 0.2 ≈ 183

Training used for learning = 914 − 183 ≈ 731

Batch Size = 32

Number of Batches per Epoch

731 / 32 ≈ 22.8

Rounded up:

= 23 batches

So the training log shows:

23/23

meaning 23 batches processed in each epoch.

Step 9: Evaluate Model

```
loss, accuracy = model.evaluate(X_test, y_test)

print("Test Accuracy:", accuracy)
```

```
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.7561 - loss: 0.5524
Test Accuracy: 0.7554585337638855
```

This checks how well the model predicts new unseen data.

Q9: What indicates overfitting?

A: High training accuracy but low test accuracy.

IN OUR MODEL

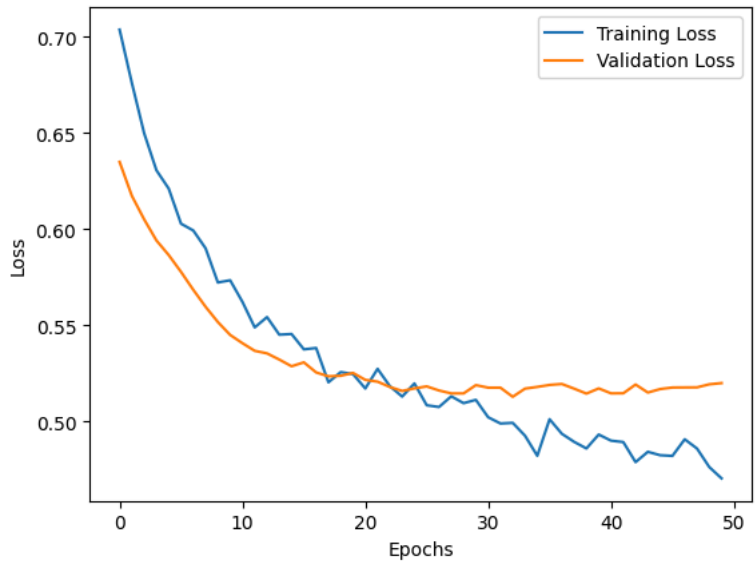
Training accuracy 76.6%

Validation accuracy 75.4%

Because the values are very close, the model is not overfitting.

Step 10: Visualize Training

```
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.legend(['Training Loss','Validation Loss'])
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.show()
```



This plot helps analyze:

convergence

overfitting

training behavior

Q11. What is convergence in neural network training?

Convergence occurs when the training process reaches a point where the loss function stops decreasing significantly and the model parameters stabilize. This means the neural network has learned the patterns in the data and further training produces little improvement.

Q12: How can we identify convergence during training?

Convergence can be identified by observing the loss curve. When the loss value gradually decreases and then stabilizes, the model is said to have converged.

Example:

Evidence of Convergence

Look at the loss trend:

Epoch 1 ⟶ loss = 0.7016

Epoch 10 ⟶ loss = 0.5884

Epoch 20 ⟶ loss = 0.5364

Epoch 50 ⟶ loss = 0.4801

Loss gradually decreases and stabilizes, meaning the model converged.

```
row = data.iloc[10]

X_sample = row.drop("quality").values.reshape(1,-1)

X_sample_scaled = scaler.transform(X_sample)

prediction = model.predict(X_sample_scaled)

print("Prediction:", prediction)
print("Actual quality:", row["quality"])
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names,
  warnings.warn(
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 161ms/step
Prediction: [[0.28339097]]
Actual quality: 5.0
```

0.28339097 ≈ 28.3% probability

Meaning the model thinks the wine belongs to class 1 with 28% confidence.

Normally we use a threshold of 0.5.

if prediction > 0.5 ⟶ class 1

if prediction < 0.5 ⟶ class 0

For your case:

0.283 < 0.5

So predicted class = 0

The neural network predicted a probability of 0.28 that the wine is a high-quality wine. Since this value is below 0.5, the model classified the wine as low quality. The actual quality score is 5, which also belongs to the low-quality class, so the prediction is correct.

```
pred_class = (prediction > 0.5).astype(int)
print(pred_class)
```

```
[[0]]
```

Q13: Why does the model output 0.28 instead of 0 or 1?

A: Because sigmoid gives a probability, not a direct class.

Q14: Why do we use 0.5 as the threshold?

A: It is the standard midpoint for binary classification probabilities.

Q15: Can the threshold be changed?

A: Yes, depending on the application (e.g., medical diagnosis).

Summary of the Exercise

This exercise demonstrates how to build and train an Artificial Neural Network (ANN) to classify wine quality using a dataset with 1143 samples and 12 features.

First, the dataset was loaded and preprocessed.

The input features (X) and target variable (y) were separated, where the target represents the wine quality.

The dataset was then divided into training and testing sets using an 80–20 split, meaning 80% of the data was used to train the model and 20% was used for testing. Additionally, 20% of the training data was used as validation data during training.

Before training the neural network, the features were scaled using a standard scaler to normalize the values and improve model performance.

Next, a Sequential ANN model was created using multiple Dense layers with ReLU activation, and a final sigmoid activation layer for binary classification.

The model was compiled using:

Optimizer: Adam

Loss function: Binary Crossentropy

Metric: Accuracy

The neural network was then trained for 50 epochs with a batch size of 32. During training, the model processed the training data in 23 batches per epoch.

As training progressed:

Loss decreased gradually

Accuracy improved

Validation accuracy stabilized around 75%

This indicates that the model successfully learned patterns in the dataset and converged without significant overfitting.

Finally, the trained model was tested on an unseen sample from the dataset to predict the wine quality class.

Final Outcome

The ANN achieved approximately 75% validation accuracy, demonstrating that neural networks can effectively classify wine quality based on chemical features.

Result-interpretation questions

1. Convergence Analysis

Question: From the training results, identify approximately at which epoch the model begins to converge. Explain how the loss and accuracy values indicate convergence.

Answer: The model starts converging around Epoch 30, where the loss stabilizes and the accuracy improvement becomes minimal.

2. Activation Function Exploration

Question:

Modify the hidden layer activation function from ReLU to tanh or sigmoid, train the model again, and compare the results. How does the activation function affect convergence and accuracy?

Answer:

ReLU: Faster convergence and good performance.

tanh: Moderate convergence.

Sigmoid: Slower learning due to possible vanishing gradient problem.

3. Optimizer Comparison

Question:

Train the model using different optimizers and record the results.

Optimizer	Training Accuracy	Validation Accuracy	Convergence Speed
Adam	High	High	Fast
SGD	Moderate	Moderate	Slow
RMSprop	High	High	Medium

4. Regularization and Accuracy Improvement

Question:

Train the neural network without Dropout and record the training and validation accuracy. Then add a Dropout layer to the model and train it again. Compare the results and comment on overfitting and model generalization.

Example modification:

Without Dropout:

Dense(16, activation='relu')

Dense(8, activation='relu')

With Dropout:

Dense(16, activation='relu')

Dropout(0.2)

Dense(8, activation='relu')

Record your observations.

Model	Training Accuracy	Validation Accuracy	Overfitting Observation
Without Dropout			
With Dropout			

5. Model Improvement Experiment

Question:

Experiment with one of the following changes and analyze its impact on model performance:

Increase the number of neurons

Increase the number of epochs

Change the optimizer

Add another hidden layer

Experiment	Change Made	Effect on Accuracy	Effect on Convergence
Baseline Model	Original ANN		
Modified Model			

Record your results.